Объектно-ориентированное программирование

4 семестр
Лекция 6.

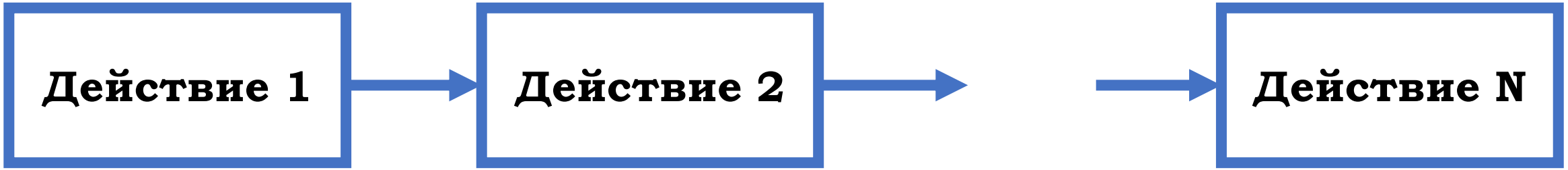
Содержание лекции:

- Асинхронное программирование
- Получение результата из асинхронного метода
- Ожидание завершения асинхронных методов

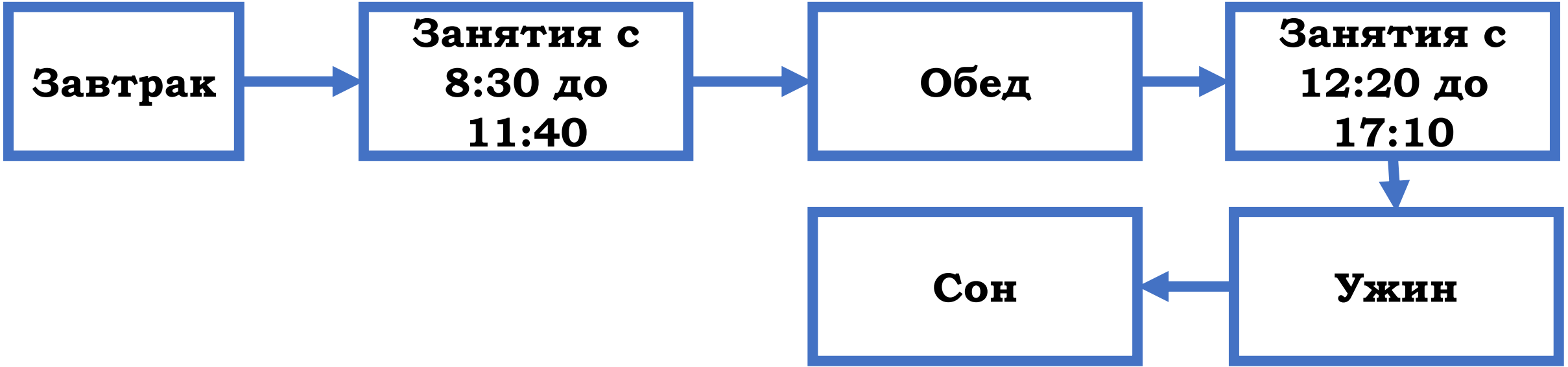
Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Синхронные и асинхронные процессы

Ключевое отличие синхронного режима от асинхронного: каждое последующее действие начинается только тогда, когда завершено предыдущее.



Синхронный режим выполнения на примере распорядка дня:

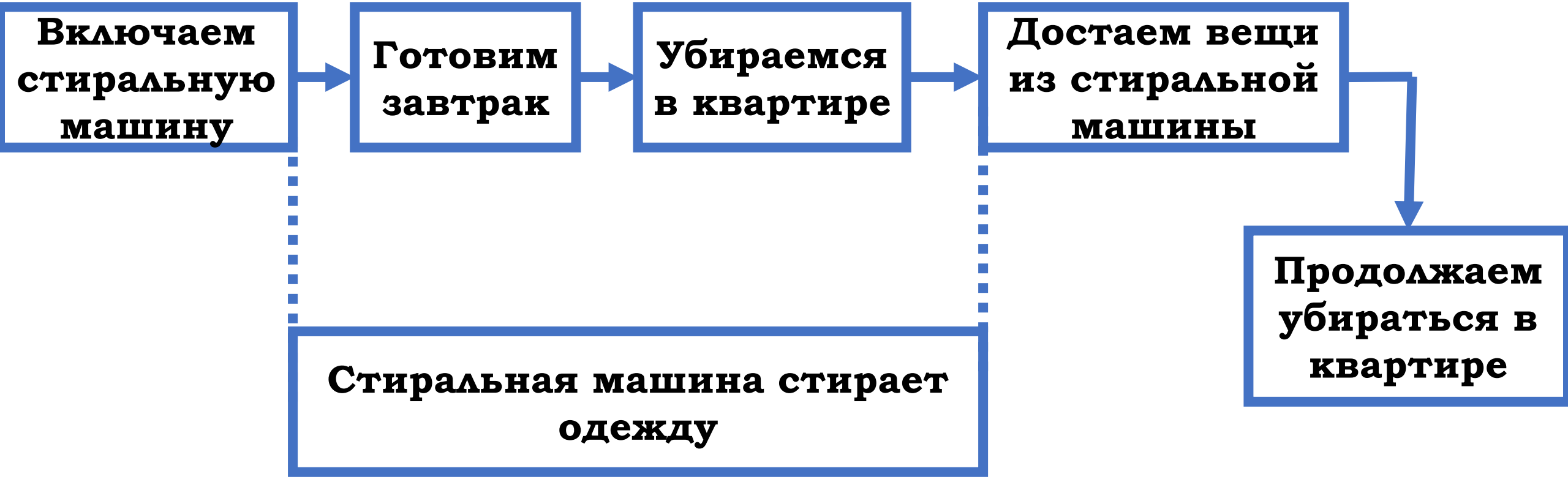


```
Print();   // 1 вызов синхронного метода
Console.WriteLine("Некоторые действия в методе Main"); // 4

void Print()
{
    Thread.Sleep(3000);     // 2 имитация продолжительной работы
    Console.WriteLine("Hello World"); // 3 Вывод в консоль
}
```

```
Hello World
Некоторые действия в методе Main
```

Пример асинхронного режима выполнения :



Асинхронное программирование

Асинхронное программирование — концепция программирования, которая заключается в том, что результат выполнения функции доступен не сразу, а через некоторое время в виде некоторого асинхронного (нарушающего обычный порядок выполнения) вызова.

Асинхронное программирование применяется при:

- обращении к сетевым ресурсам;
- чтении-записи больших файлов;
- обращении к базе данных;
- выполнении трудоемких вычислений.

Ключевыми для работы с асинхронными вызовами в C# являются два оператора: **async** и **await**, цель которых - упростить написание асинхронного кода. Они используются вместе для создания асинхронного метода.

```
async Task Имя_метода()
{
   // Инструкции, выполняющиеся в основном потоке
   await Task.Run(ExampleMethod()); // Выполняется асинхронно
   // Инструкции, выполняющиеся после завершения ExampleMethod
}
```

Асинхронный метод обладает следующими **признаками**:

1. В заголовке метода используется модификатор **async**
2. Метод содержит одно или несколько выражений **await**
3. В качестве возвращаемого типа используется один из следующих:
   - **void**
   - **Task**
   - **Task<T>**
   - **ValueTask<T>**

Также стоит отметить, что слово **async**, которое указывается в определении метода, **НЕ** делает автоматически метод асинхронным. Оно лишь указывает, что данный метод может содержать одно или несколько выражений **await**.

```
var printTask = PrintAsync();   // 1 вызов асинхронного метода
Console.WriteLine("Некоторые действия в методе Main"); // 3
await printTask;
//6 ...
// определение асинхронного метода
async Task PrintAsync()
{
    // 2
    Console.WriteLine("Начало метода PrintAsync");
    await Task.Run(Print); // 4 Выполняется асинхронно
    Console.WriteLine("Конец метода PrintAsync"); // 5
}
void Print()
{
Thread.Sleep(3000);
Console.WriteLine(
  "Hello World");
}
```

```
Начало метода PrintAsync
Некоторые действия в методе Main
Hello World
Конец метода PrintAsync
```

Преимущества асинхронности

Рассмотрим синхронный метод, **сколько времени необходимо для выполнения данной программы?**

```
PrintName("Петр");
PrintName("Владимир");
PrintName("Сергей");

void PrintName(string name)
{
    Thread.Sleep(3000);     // имитация продолжительной работы
    Console.WriteLine(name);
}
```

```
Петр
Владимир
Сергей
```

Изменим в программе синхронный метод PrintName на асинхронный:

```
await PrintNameAsync("Петр");
await PrintNameAsync("Владимир");
await PrintNameAsync("Сергей");

// определение асинхронного метода
async Task PrintNameAsync(string name)
{
    await Task.Delay(3000);     // имитация продолжительной работы
    Console.WriteLine(name);
}
```

```
Петр
Владимир
Сергей
```

Сколько времени уйдет на выполнение данных методов?

Оптимизируем программу:

```
var task1 = PrintNameAsync("Петр");
var task2 = PrintNameAsync("Владимир");
var task3 = PrintNameAsync("Сергей");
await task1;
await task2;
await task3;
// определение асинхронного метода
async Task PrintNameAsync(string name)
{
    await Task.Delay(3000);     // имитация продолжительной работы
    Console.WriteLine(name);
}
```

```
Владимир
Сергей
Петр
```

Первый пример выполнялся 9 секунд в основном потоке;
Второй пример выполнялся 9 секунд в другом потоке и основной поток оставался свободным;
Третий пример выполняется 3 секунды в отдельных потоках, основной поток также свободен.

Определение асинхронного лямбда-выражения:

```
// асинхронное лямбда-выражение
Func<string, Task> printer = async (message) =>
{
    await Task.Delay(1000);
    Console.WriteLine(message);
};

await printer("Привет Мир!"); // 1 вызываем 1-й метод
await printer("Конец"); // 2 вызываем 2-й метод
```

Сколько времени будет выполняться программа?

```
Привет Мир!
Конец
```

Чтобы методы выполнялись одновременно необходимо написать:

```
// асинхронное лямбда-выражение
Func<string, Task> printer = async (message) =>
{
    await Task.Delay(1000);
    Console.WriteLine(message);
};
var task1 = printer("Привет Мир!"); // 1 вызываем 1-й метод
var task2 = printer("Конец"); // 2 вызываем 2-й метод
Task.WaitAll(task1, task2); // Ждем завершения
```

Сколько времени будет выполняться программа?

```
Конец
Привет Мир!
```

Получение результата из асинхронного метода

В качестве возвращаемого типа в асинхронном методе должны использоваться типы void, Task, Task<T> или ValueTask<T>.

void

```
PrintAsync("Привет Мир!");
PrintAsync("Привет математика!");

Console.WriteLine("Конец");
// ждем завершения задач
await Task.Delay(3000);

// определение асинхронного метода
async void PrintAsync(string message)
{
    await Task.Delay(1000);     // имитация продолжительной работы
    Console.WriteLine(message);
}
```

```
Конец
Привет математика!
Привет Мир!
```

Task

```
var task1 = PrintAsync("Привет Мир!");
var task2 = PrintAsync("Привет математика!");

await task1;
await task2;

// определение асинхронного метода
async Task PrintAsync(string message)
{
    await Task.Delay(1000);     // имитация продолжительной работы
    Console.WriteLine(message);
}
```

```
Привет математика!
Привет Мир!
```

Task<T>

```
var square5 = SquareAsync(5);
var square6 = SquareAsync(6);
Console.WriteLine("Остальные действия в методе Main");
int n1 = await square5;
int n2 = await square6;
Console.WriteLine($"n1={n1}  n2={n2}"); // n1=25  n2=36
async Task<int> SquareAsync(int n)
{
    await Task.Delay(3000); // имитация продолжительной работы
    var result = n * n;
    Console.WriteLine($"Квадрат числа {n} равен {result}");
    return result;
}
```

```
Остальные действия в методе Main
Квадрат числа 6 равен 36
Квадрат числа 5 равен 25
n1=25  n2=36
```

Task.WhenAll и Task.WhenAny

Асинхронный метод может содержать множество выражений await. Когда система встречает в блоке кода оператор await, то выполнение в асинхронном методе останавливается, пока не завершится асинхронная задача. После завершения задачи управление переходит к следующему оператору await и так далее. Это позволяет вызывать асинхронные задачи последовательно в определенном порядке.

```
var task1 = PrintAsync("Привет Мир!");
var task2 = PrintAsync("Привет математика!");
var task3 = PrintAsync("Конец");

await task1;
await task2;
await task3;
```

.NET позволяет упростить отслеживание выполнения набора задач с помощью метода Task.WhenAll.

```
// определяем и запускаем задачи
var task1 = PrintAsync("Привет C#");
var task2 = PrintAsync("Привет Мир!");
var task3 = PrintAsync("Конец");
// ожидаем завершения всех задач
await Task.WhenAll(task1, task2, task3);
async Task PrintAsync(string message)
{
// имитация продолжительной операции
    await Task.Delay(2000);
    Console.WriteLine(message);
}
```

```
Привет C#
Конец
Привет Мир!
```

Если необходимо дождаться, когда будет выполнена хотя бы одна задача из некоторого набора задач, то применяется метод Task.WhenAny().

```
// определяем и запускаем задачи
var task1 = PrintAsync("Привет C#");
var task2 = PrintAsync("Привет Мир!");
var task3 = PrintAsync("Конец");
// ожидаем завершения хотя бы одной задачи
await Task.WhenAny(task1, task2, task3);
async Task PrintAsync(string message)
{
    await Task.Delay(new Random().Next(1000, 2000));
     // имитация продолжительной операции
    Console.WriteLine(message);
}
```

```
Конец
```

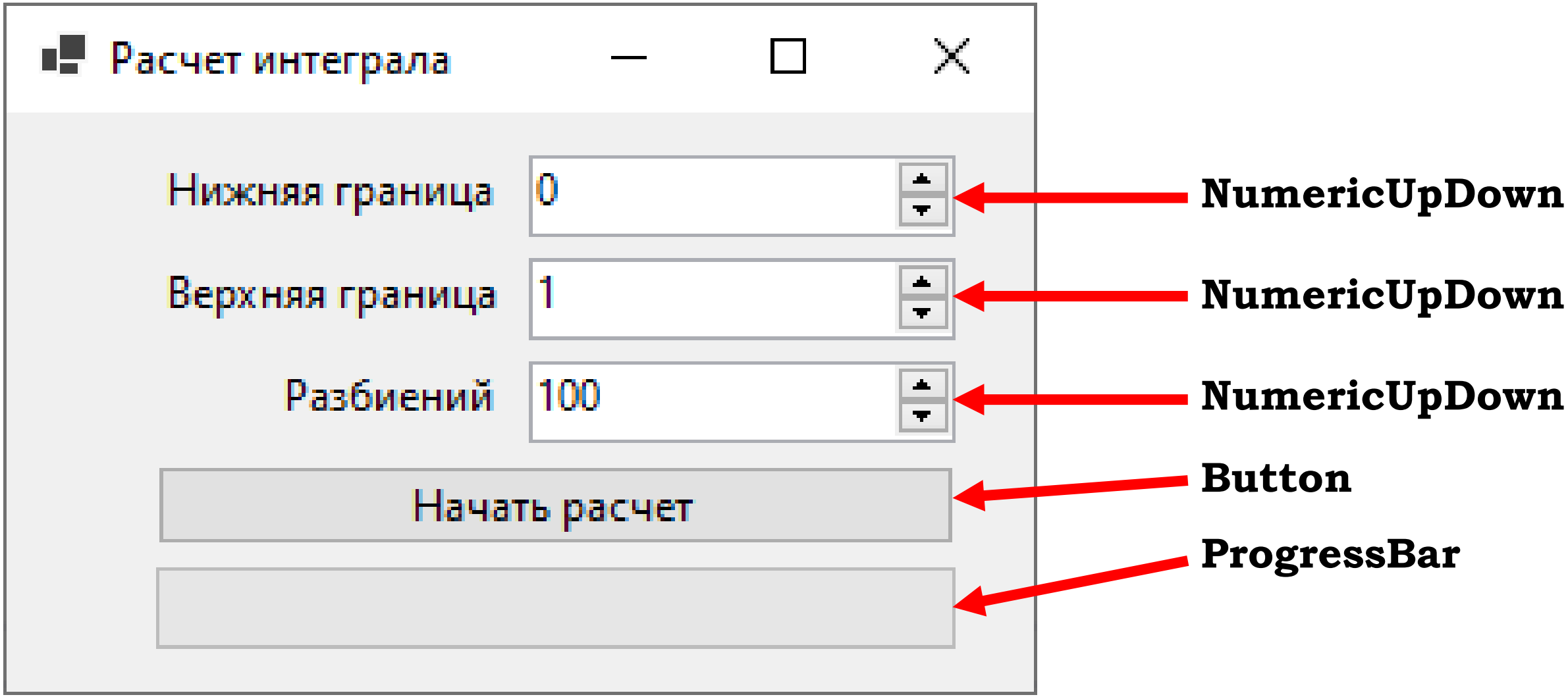
Задачи, передаваемые в Task.WhenAll и Task.WhenAny, могут возвращать некоторое значение. В этом случае из методов Task.WhenAll и Task.WhenAny можно получить массив, который будет содержать результаты задач:

```
var task1 = SquareAsync(4); // определяем и запускаем задачи
var task2 = SquareAsync(5);
var task3 = SquareAsync(6);
// ожидаем завершения всех задач
int[] results = await Task.WhenAll(task1, task2, task3);
foreach (int result in results) // выводим результаты:
    Console.WriteLine(result);
async Task<int> SquareAsync(int n) {
    await Task.Delay(1000);
    return n * n;
}
```

```
16
25
36
```

Пример асинхронного расчета интеграла

Создадим графическое приложения для расчета интеграла.



Создадим метод вычисления интеграла с помощью метода трапеции:

```
private decimal Integrate(decimal a, decimal b, long N)
{
            var h = (b - a) / N;
            var sum = 0.0M;

            for (long i = 0; i < N; i++)
            {
                var x = a + i * h;
                sum += (f(x) + f(x + h)) * h / 2;
            }

            return sum;
}

 private decimal f(decimal x) => x * x;
```

Создадим метод вычисления интеграла с помощью метода трапеции:

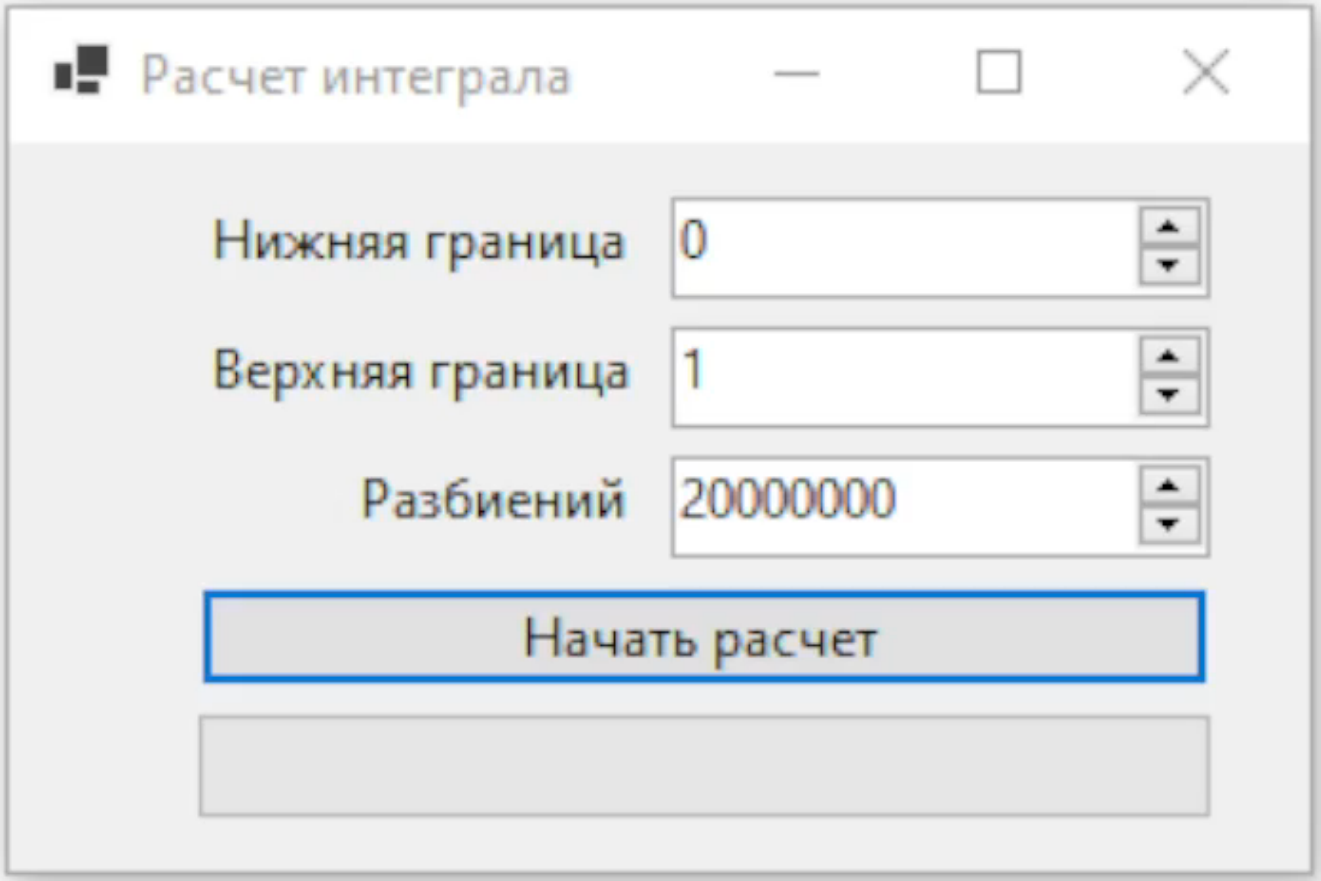
```
private async void calcButton_Click(object sender, EventArgs e)
{
   var a = numericUpDown1.Value;
   var b = numericUpDown2.Value;
   var N = (long)numericUpDown3.Value;

   task = Task.Run(() => Integrate(a, b, N));

   var integral = await task;

   MessageBox.Show($"Интеграл равен {integral:0.000}", "Ответ");
}
```

Запустим расчет, т.к. вызов метода асинхронный и вычислением интеграла занимается отдельный поток, то программа не зависнет.



Для отображение прогресса изменим программу. Добавим метод отображения прогресса:

```
private void UpdateProgress(decimal pcnt)
        {
            if (Math.Abs(pcnt - progressBar.Value) >= 1)
                Invoke(() => progressBar.Value = (int)pcnt);
        }
```

Добавим делегат для хранения метода отображения прогресса:

```
public delegate void Observer(decimal pcnt);
```

Изменим сигнатуру метода расчета интеграла:

```
private decimal Integrate(
            decimal a, decimal b,
            long N, Observer? observer = null)
```

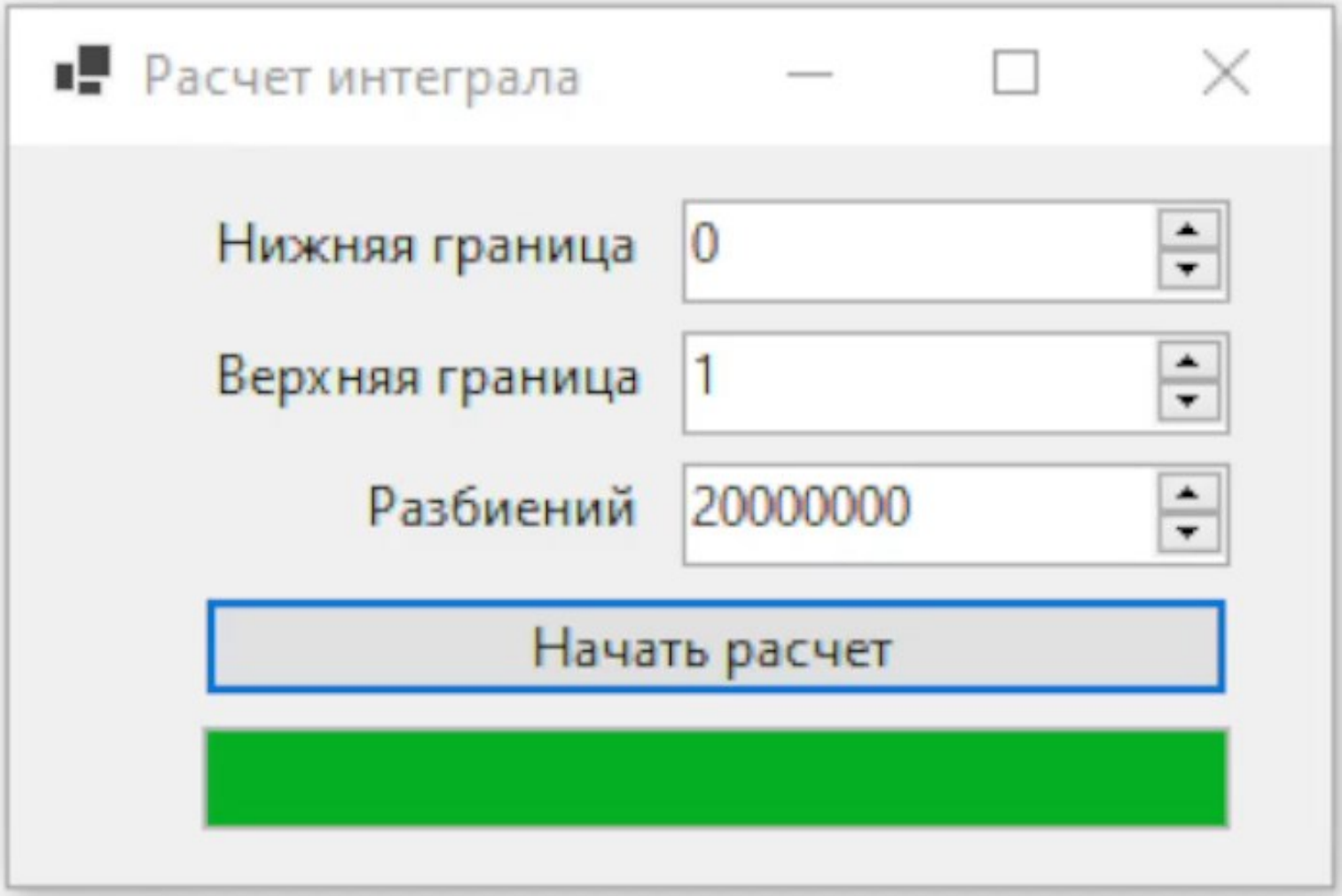
Добавим вызов метода отображения прогресса при расчете интеграла:

```
private decimal Integrate(
            decimal a, decimal b,
            long N, Observer? observer = null)
        {
            var h = (b - a) / N;
            var sum = 0.0M;

            for (long i = 0; i < N; i++)
            {
                var x = a + i * h;
                sum += (f(x) + f(x + h)) * h / 2;
                observer?.Invoke((decimal)i * 100 / (N - 1));
            }

            return sum;
        }
```

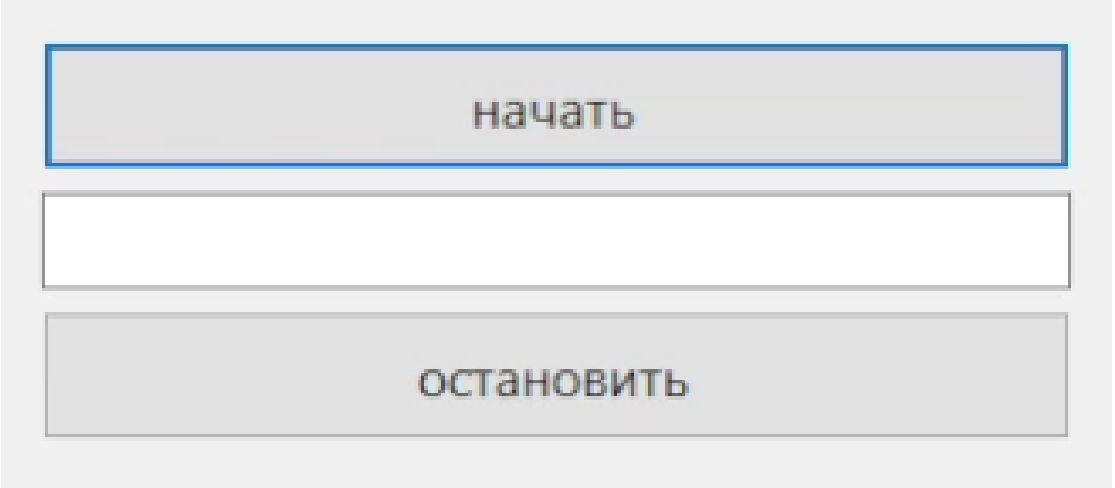
Запустим расчет:



Задание

В соответствии с вариантом разработать приложение для асинхронного расчета суммы ряда. Не использовать таймер и т.п. Необходимо использовать: Task<decimal>, async / await, CancellationTokenSource и CancellationToken. Сумма ряда должна считаться до бесконечности, пока задача не будет прервана с помощью метода Cancel() у CancellationTokenSource.

Видео-пример работы программы:



Задержка асинхронной операции и Task.Delay

В асинхронных методах для остановки метода на некоторое время можно применять метод Task.Delay(). В качестве параметра он принимает количество миллисекунд в виде значения int, либо объект TimeSpan, который задает время задержки:

```
await PrintAsync();   // вызов асинхронного метода
Console.WriteLine("Некоторые действия в методе Main");
// определение асинхронного метода

async Task PrintAsync()
{
// имитация продолжительной работы
    await Task.Delay(3000);
//или await Task.Delay(TimeSpan.FromMilliseconds(3000));
    Console.WriteLine("Hello World");
}
```

```
Hello World
Некоторые действия в методе Main
```

Причем метод Task.Delay сам по себе представляет асинхронную операцию, поэтому к нему применяется оператор await.